

T1.4.D — /api/dashboard-data server-side

Proxy landing → backend con merge Stripe SDK · ningún ID del cliente

Owner: Celestino · 2026-05-03 · Branch `pr/t1.4.d-dashboard-data` · Commit `732f247`

Status: Branch pusheado a GitHub. Build OK · 16/16 static pages · /api/dashboard-data registrado como *f* (server-rendered). tsc 0 errores · lint sin nuevos. Secrets no aparecen en bundle estático del cliente. **NO mergeado a main.**

1. Archivos modificados

Archivo	Δ líneas	Tipo	Rol
landing/lib/internal-bridge.ts	+141 / -0	M	fetchUsuarioResumen() + tipos + shortId() helper
landing/app/api/dashboard-data/route.ts	+150 / -0	A	GET handler server-only
TOTAL	+291 / -0	2 archivos	

2. Formato final del endpoint

Ruta

GET /api/dashboard-data
(no acepta body ni query params para identificar usuario)

Códigos HTTP

Status	Cuándo
401	Sin sesión NextAuth (auth()) retornó null
403	Sesión sin subscriptionId (caso raro: sub canceled hace mucho)
404	Backend respondió 404 — subscription_not_found
502	Backend respondió 401 (secret desincronizado) o cualquier otro no-2xx
504	Timeout llamando al backend (>8s)
200	Caso feliz: JSON con datos del backend + overrides Stripe + email

Body response (200)

```
{
  "usuario": {
    "id": "sub_xxx",
    "email": "user@example.com", // de session.user.email
    "telefono": "14076936023"
  },
  "creditos": {
    "saldo_actual": 170,
    "creditos_mensuales": 100,
    "ultimo_movimiento": {...}
  },
}
```

```
"suscripcion": {
  "estado": "active",           // del backend
  "plan": "premium",           // del backend
  "stripe_customer_id": "cus_xxx",
  "stripe_subscription_id": "sub_xxx",
  "current_period_end": 1717545600, // de Stripe SDK
  "cancel_at_period_end": false,    // de Stripe SDK
  "actualizado": "2026-05-02T..."
},
"transacciones_recientes": [...], // del backend
"resumen": {
  "puede_cancelar": true,
  "dashboard_ready": true
}
}
```

3. Cómo se obtiene subscriptionId

Guardrail #1 (server-side identifiers): el subscriptionId se toma **únicamente** de la sesión NextAuth server-side. **NUNCA** del request del cliente (ni body, ni query, ni headers).

```
export async function GET() {
  const session = await auth();
  if (!session) {
    return NextResponse.json({error: "unauthenticated"}, {status: 401});
  }

  // 2. Tomar subscriptionId SOLO de la sesión server-side.
  // Guardrail T1.4.D: NUNCA del request del cliente.
  const subscriptionId = (session as {subscriptionId?: string})
    .subscriptionId;
  const sessionEmail = session.user?.email ?? null;

  if (!subscriptionId) {
    return NextResponse.json(
      {error: "no_subscription_in_session"},
      {status: 403},
    );
  }
  // ...
}
```

El handler es **GET**, no acepta JSON body. Aunque el cliente intentara mandar un subscription_id en query string o headers, el código no los lee. La sesión NextAuth se lee con *auth()* que valida la cookie JWT firmada del usuario.

4. Cómo se firma HMAC

Reusa el patrón de T1.3.E: HMAC-SHA256 del body crudo con INTERNAL_BRIDGE_SECRET, header *X-Internal-Signature*. La lógica está en *fetchUsuarioResumen* de *internal-bridge.ts*:

```
export async function fetchUsuarioResumen(
  subscriptionId: string,
): Promise {
  const backendUrl = process.env.BACKEND_URL?.trim();
  const secret = process.env.INTERNAL_BRIDGE_SECRET?.trim();

  if (!backendUrl) return {ok: false, error: "backend_url_missing"};
  if (!secret) return {ok: false, error: "internal_bridge_secret_missing"};

  const body = JSON.stringify({subscription_id: subscriptionId});
  const signature = crypto
    .createHmac("sha256", secret)
    .update(body, "utf8")
    .digest("hex");

  const url = `${backendUrl.replace(/\/+$/, "")}/internal/usuario-resumen`;
  const res = await fetch(url, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "X-Internal-Signature": signature,
    },
    body, signal: controller.signal,
  });
  // ... mapping de res.status a {ok, status, error}
}
```

El body que firmamos es **exactamente el body que enviamos** (misma referencia de string). Timeout 8s vía AbortController.

5. Cómo se consulta Stripe

`fetchStripeOverrides` en `route.ts` hace `subscriptions.retrieve(subscriptionId)`. En **stripe@22** el campo `current_period_end` se movió del root del Subscription al primer subscription item.

```
async function fetchStripeOverrides(subscriptionId: string) {
  if (!process.env.STRIPE_SECRET_KEY) return DEFAULT;
  try {
    const sub = await getStripe().subscriptions.retrieve(
      subscriptionId,
    );
    // current_period_end vive en items.data[0] en stripe@22.
    const firstItem = sub.items?.data?.[0];
    return {
      current_period_end: typeof firstItem?.current_period_end === "number"
        ? firstItem.current_period_end : null,
      cancel_at_period_end: Boolean(sub.cancel_at_period_end),
      status_stripe: sub.status ?? null,
    };
  } catch (err) {
    // Log con shortId(); no propagar el error al cliente.
    return DEFAULT;
  }
}
```

Si Stripe falla (red, no configurado en dev, etc.), devolvemos defaults (`period_end=null`, `cancel=false`). El backend response queda válido, solo el dashboard no mostrará la fecha de renovación.

Paralelización

Backend y Stripe se llaman en paralelo con *Promise.all* para no apilar latencias. La total esperada es `max(backend, stripe)`, no su suma.

6. Guardrails respetadas (Dona Control review T1.4.C)

Guardrail	Cómo se cumple
#1: subscriptionId server-side only	Se lee SOLO de <code>session.subscriptionId</code> . El handler es GET sin body, no lee query params para id
#2: no loguear subscription_id completo	Helper <code>shortId()</code> trunca a 'sub_XXXXXXX...abcd' antes de pasar a <code>console.error</code> . Aplicado en <code>fetchU</code>
No exponer INTERNAL_BRIDGE_SECRET	Solo se lee dentro de <code>fetchUsuarioResumen</code> (server-only). Verificado: ausente en <code>.next/static/</code> .
No exponer STRIPE_SECRET_KEY	Solo en <code>getStripe()</code> (server-only). Misma verificación.
No filtrar errores crudos del backend	Códigos semánticos: <code>subscription_not_found</code> , <code>backend_auth_error</code> , <code>backend_timeout</code> , <code>backend_un</code>

7. Comandos ejecutados y resultado

Comando	Resultado
npx tsc --noEmit	0 errores tras ajustar current_period_end → items.data[0]
npm run lint	16 problems · todos preexistentes (any types en otros archivos). 0 nuevos por T1.4.D.
npm run build	✓ Compiled successfully · 16/16 static pages · /api/dashboard-data registrado como f (dynamic, server-rendered)
grep secrets en .next/static/	0 matches · INTERNAL_BRIDGE_SECRET y STRIPE_SECRET_KEY no aparecen en bundle del cliente
grep secrets en .next/server/	Sí aparecen (esperado: el código server-side lee process.env)

8. Riesgos abiertos

Riesgo	Severidad	Mitigación
Sesión NextAuth comprometida → atacante consume dashboard-data	Bajo	Solo accesible con SU PROPIO data. Mitigación primaria es T1.4.B (password valid)
INTERNAL_BRIDGE_SECRET filtrado	Alta	Rotar en Vercel y Render simultáneamente (mismo riesgo que T1.3.D/E).
STRIPE_SECRET_KEY filtrado	Alta	Rotar en Vercel + Render. Stripe permite revocación inmediata.
Stripe SDK retrieve falla (red, rate-limit)	Bajo	Manejado: devolvemos defaults. Dashboard se renderiza sin period_end pero n
Backend timeout > 8s	Bajo	Devolvemos 504 al cliente; el dashboard puede reintentar.
Sin rate limiting en /api/dashboard-data	Bajo	Solo accesible con sesión válida. Sesión sirve como rate limit indirecto. Mitigab

9. Próximos pasos para T1.4.E (UI dashboard)

T1.4.E es la UI: `landing/app/dashboard/dashboard-client.tsx` consume `/api/dashboard-data` y muestra:

- **Saldo actual** grande (ej. "170 créditos") + indicador de créditos mensuales del plan.
- **Plan activo** real desde backend (premium / pro), no más comparación con `priceId` del env.
- **Próxima renovación** formateada (`current_period_end` → fecha humana).
- **cancel_at_period_end** visible si aplica con texto tipo "Se cancela el DD/MM".
- **Historial**: tabla con últimas 10 transacciones (delta, razón, saldo resultante, fecha).
- **Estados de carga**: skeleton mientras carga, mensaje de error legible si `dashboard-data` falla.

Importante para T1.4.E: el cliente solo conoce los datos que aparecen en el response de `/api/dashboard-data`. NO añadir fetches a Stripe SDK ni al backend desde el browser. Si T1.4.E necesita más datos, extender T1.4.D.

T1.4.F (después de T1.4.E)

Cancelación segura: cambiar `stripe.subscriptions.cancel()` por `stripe.subscriptions.update({cancel_at_period_end: true})` en `/api/cancel-subscription`. UX: "se cancela el DD/MM, hasta esa fecha podés seguir usando los créditos".